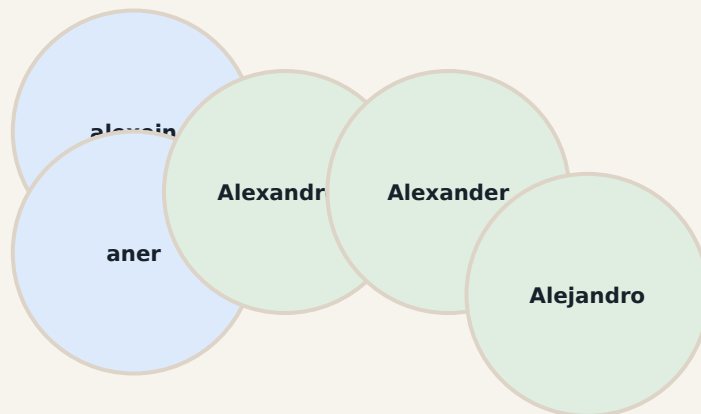


NOMINA

Developer guide for the etymological name graph prototype



Prepared for handing the prototype to another developer

Version 1.0 - April 2026

Table of contents

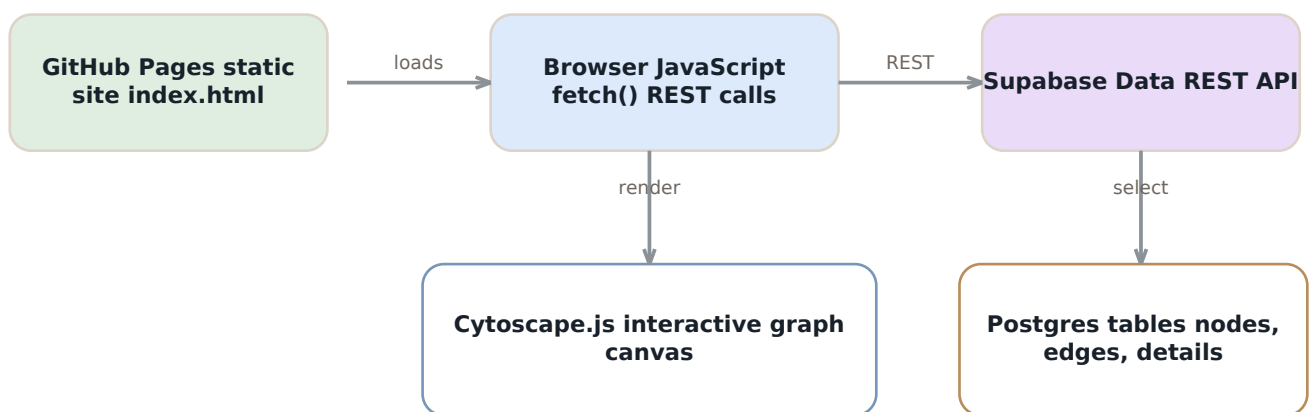
1. Executive overview
2. What the prototype does
3. System architecture
4. UI layout and rendering model
5. Database design principle
6. Table-by-table schema guide
7. Edge and lineage rules
8. How the code queries the database
9. How to add a new name
10. How to update the frontend code
11. Security and deployment notes
12. Troubleshooting
13. Developer handover checklist
14. References

This guide documents the current Nomina prototype: a static GitHub Pages website that reads graph data from Supabase and renders an interactive etymology map in the browser.

1. Executive overview

Nomina is a browser-based etymological dictionary of names. The important design principle is that the graph shows structure while the side panel tells the story. The database stores names, word roots, concepts, edges, language metadata, sources, and curated lineages. The frontend is intentionally simple: a single static HTML file queries Supabase via REST and renders the graph with Cytoscape.js.

The current site is suitable for a prototype because GitHub Pages can publish static files from a repository branch, Supabase exposes browser-callable REST endpoints from the database schema, and Cytoscape.js is a JavaScript graph visualisation library for interactive network diagrams. See the references section for the official documentation.

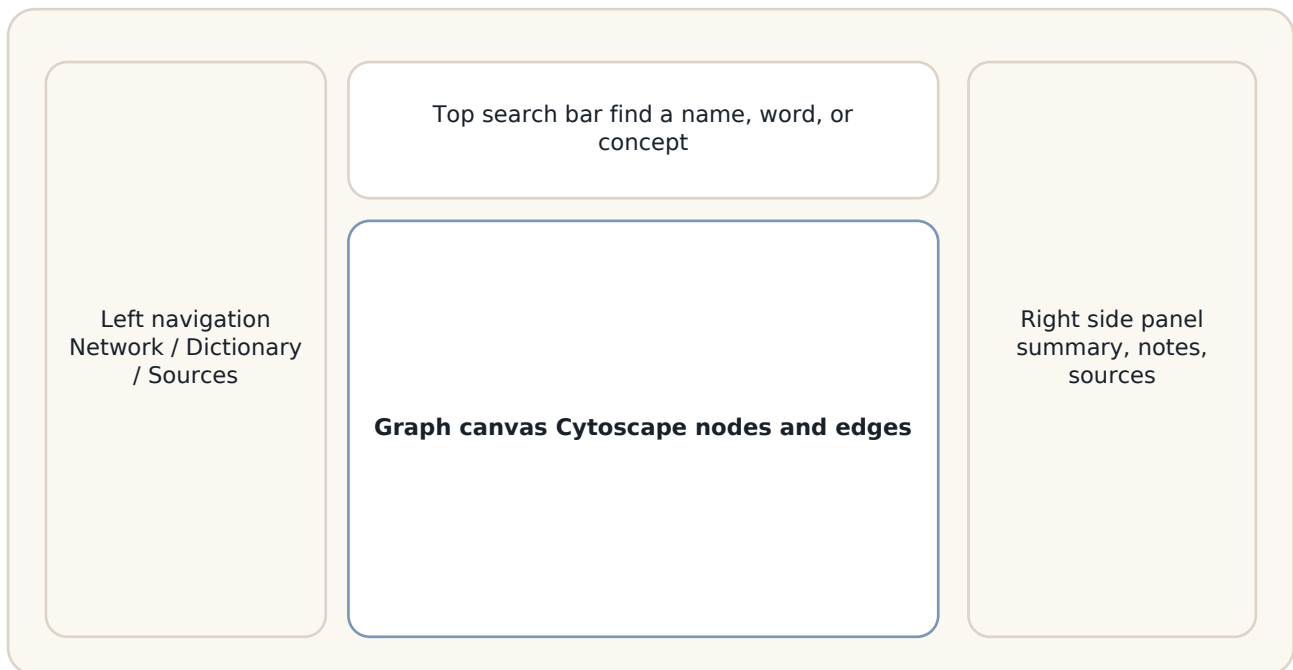


Developer intent

- A developer should be able to add data without changing the frontend code, provided new rows follow the same schema and lineage rules.
- Every graph bubble is a row in nodes. Name-specific, word-specific, and concept-specific details live in their own detail tables.
- The website loads a selected node, its curated lineage, connected edges, connected nodes, and the side panel details.
- The public website should only use the Supabase publishable key. Never put a service role key in frontend code.

2. What the prototype does

The prototype begins by loading the name node with slug alexander. It then looks for that node's primary lineage, pulls the nodes listed in path_nodes, expands the graph by fetching edges connected to that path, and renders the result on the graph canvas. Searching for another slug repeats the same process for the selected node.



Visible user flow

User action	Frontend behaviour	Database tables used
Open page	Loads slug alexander by default and renders graph.	nodes, lineages, edges, names, words, concepts, languages, sources
Search	Queries nodes by slug/display_label using ilike matching.	nodes
Click search result	Re-centres local graph around that selected node.	nodes, lineages, edges
Click graph node	Updates side panel with detail row and sources.	names/words/concepts, languages, sources
Click edge	Shows relationship explanation and certainty.	edges

3. System architecture

The current version deliberately avoids build tools, environment variables, and server-side code. The application is a plain static site: index.html contains the layout, CSS, and JavaScript. Browser JavaScript calls the Supabase REST API using fetch(). Supabase returns JSON rows. Cytoscape.js converts those rows into nodes and edges on the canvas.

Layer	Current implementation	Why it is used
Hosting	GitHub Pages	Easy static hosting. Push index.html and publish from the repository root or docs folder.
Database/API	Supabase Postgres + Data REST API	Database rows are exposed through REST endpoints from the public schema.
Graph renderer	Cytoscape.js loaded from CDN	Interactive browser graph: pan, zoom, click nodes and edges.
Frontend	Single-file vanilla HTML/CSS/JS	No local setup; easy for prototype handoff.
Security control	RLS policies and publishable key	The publishable key identifies the client; RLS decides what can be read.

Architectural rule: keep the frontend dumb and the data meaningful. If the graph looks wrong, check the data first: node slugs, lineage path_nodes, and duplicate edges.

4. UI layout and rendering model

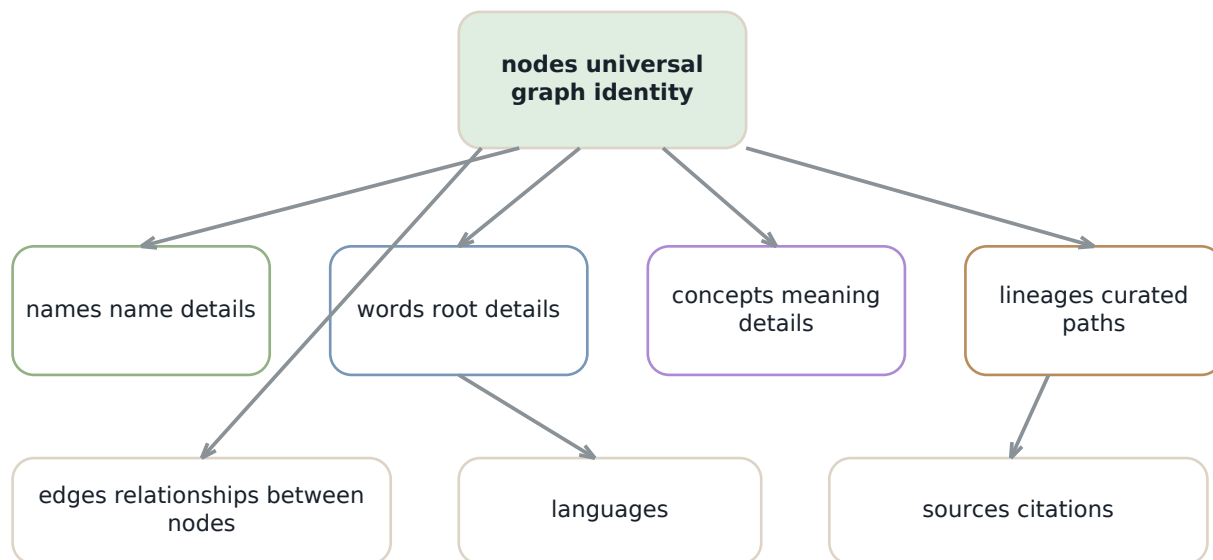
The UI is divided into three functional areas: the left navigation/legend, the central Cytoscape graph canvas, and the right-hand details panel. The central canvas is driven by nodes and edges. The right panel is driven by the detail table corresponding to the selected node_type.

UI area	Purpose	Main code functions
Left sidebar	Branding, navigation placeholder, legend for node colours.	Static HTML/CSS
Top search	Search nodes by slug or display_label.	runSearch(), db()
Graph canvas	Render local graph around selected node.	loadGraphByNode(), renderGraph(), makePositions()
Right side panel	Show node details, language, and sources.	selectNode(), loadDetailsForNode(), renderDetails(), renderSources()

Colour mapping

Node/edge kind	Current colour/style	Meaning
name node	soft green circle	A proper name or name form.
word node	soft blue circle	A root, stem, or source word.
concept node	soft purple circle	A meaning concept, such as protection or man/mankind.
selected node	dark navy larger circle	The current focus node.
meaning_of edge	purple dotted line	Concept -> word meaning link.
element_of edge	blue line	Word -> name element contribution.
evolves_to edge	dark arrowed line	Directional historical/transmission step.
variant_of edge	green dashed line without arrow	Direct variant, short form, or gendered form.

5. Database design principle



The nodes table is the universal identity table. It holds the objects that can appear as graph bubbles. The detail tables do not need their own slug because they are linked to `nodes.node_id`. Edges connect node IDs. Lineages store curated paths through nodes. Sources and languages provide supporting metadata.

Core principle

- Graph identity belongs in nodes.
- Name details belong in names.
- Word/root details belong in words.
- Meaning details belong in concepts.
- Relationships belong in edges.
- Curated evolutionary paths belong in `lineages.path_nodes`.
- Sources are stored once in sources and referenced by `source_ids` JSON arrays.
- Languages are stored once in languages and referenced by language foreign keys.

A node may be a name, word, or concept. A node detail row should use the same `node_id` as its parent node. Do not create a new UUID manually for a detail row; the UUID comes from `nodes.id`.

6. Table-by-table schema guide

The following fields reflect the current prototype design. Where a field is described as JSONB, keep the value as valid JSON, not comma-separated plain text.

nodes

Field	Type	Purpose
id	uuid, primary key	Generated by Supabase/Postgres. Universal graph node ID.

Field	Type	Purpose
node_type	text	One of name, word, concept. Used by rendering and detail lookup.
slug	text	Stable readable lookup key, unique with node_type. Example: alejandro.
display_label	text	Label shown on the graph. Example: Alejandro.
canonical_key	text	Simplified matching key. Example: alexander.
description_short	text	Short preview for search and side panel subtitle.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Last update timestamp.

names

Field	Type	Purpose
node_id	uuid, PK/FK -> nodes.id	Links this detail row to a name node.
display_name	text	Full name display text.
original_script	text	Original script if different. Example: Ἀλέξανδρος or اردنكس.
transliteration	text	Latin transliteration.
primary_language_id	uuid, FK -> languages.id	Primary language/culture of this name form.
gender_usage	text	masculine, feminine, unisex, etc.
short_summary	text	One or two sentence summary.
long_summary	text	Main side panel explanation.
literary_notes	text	Myth, literature, historic people, premodern context.
cultural_notes	text	Cultural usage and transmission notes.
pronunciation_notes	text	Optional pronunciation guidance.
certainty_notes	text	Uncertainty and source caveats.
source_ids	jsonb	Array of source UUID strings.
created_at / updated_at	timestamp	Audit timestamps.

words

Field	Type	Purpose
node_id	uuid, PK/FK -> nodes.id	Links this detail row to a word node.
display_text	text	Root/word displayed in side panel.
language_id	uuid, FK -> languages.id	Language of the word/root.
original_script	text	Original script. Example: ἀλέξω / ἀλέξειν.
transliteration	text	Romanised form. Example: alexō / alexein.
word_type	text	noun, verb, stem, source element, etc.
literal_meaning	text	Meaning of the word/root.
grammar_notes	text	Stem/case/compound notes.
additional_notes	text	Extra notes. Important: this field is not called notes.
source_ids	jsonb	Array of source UUID strings.
created_at / updated_at	timestamp	Audit timestamps.

concepts

Field	Type	Purpose
node_id	uuid, PK/FK -> nodes.id	Links this detail row to a concept node.
display_text	text	Concept label. Example: protection.
canonical_key	text	Normalised concept key.
description	text	Side panel explanation.
source_ids	jsonb	Array of source UUID strings.
created_at / updated_at	timestamp	Audit timestamps.

edges

Field	Type	Purpose
id	uuid, primary key	Generated edge ID.
from_node_id	uuid, FK -> nodes.id	Source node. Direction matters for directed edge types.
to_node_id	uuid, FK -> nodes.id	Target node.
edge_type_code	text, FK -> edge_types.code	Relationship kind: meaning_of, element_of, evolves_to, variant_of.
certainty	numeric	Confidence score, usually 0-1.
explanation	text	Plain-English explanation of the relationship.
source_ids	jsonb	Array of source UUID strings.
created_at / updated_at	timestamp	Audit timestamps.

edge_types

Field	Type	Purpose
code	text, primary/unique key	Machine-readable edge code. Used by frontend CSS/style selectors.
label	text	User-readable label.

languages

Field	Type	Purpose
id	uuid, primary key	Generated language ID.
name	text	Display name. Example: Ancient Greek.
slug	text, unique	Readable lookup key. Example: ancient-greek.
alternative_names	jsonb	Array of alternate labels.
language_family	text	Family/classification. Example: Romance / Indo-European.
period_label	text	Historical period or broad period.
region_note	text	Where it is/was used.
additional_notes	text	Extra notes.
source_ids	jsonb	Array of source UUID strings.
created_at / updated_at	timestamp	Audit timestamps.

lineages

Field	Type	Purpose
id	uuid, primary key	Generated lineage ID. A node can have more than one lineage.
node_id	uuid, FK -> nodes.id	The owner/focus node of the lineage.

Field	Type	Purpose
lineage_type	text	primary, alternative, disputed, etc.
title	text	User-facing lineage title.
summary	text	Short explanation of the path.
path_nodes	jsonb	Nested array of node IDs. Each inner array is a step group.
certainty	numeric	Confidence score, usually 0-1.
source_ids	jsonb	Array of source UUID strings.
created_at / updated_at	timestamp	Audit timestamps.

sources

Field	Type	Purpose
id	uuid, primary key	Generated source ID.
title	text	Source title. Example: Behind the Name - Alejandro.
author	text	Author or organisation.
source_type	text	dictionary, website, academic article, etc.
url	text	URL if online.
citation_text	text	Human-readable citation shown in source list.
notes	text	Why this source was used or reliability note.
created_at / updated_at	timestamp	Audit timestamps.

7. Edge and lineage rules

The edge table should stay sparse and meaningful. The lineage can show a full path, but that does not mean every related name needs a direct edge to every other related name. The graph should avoid shortcut edges that make the map noisy.

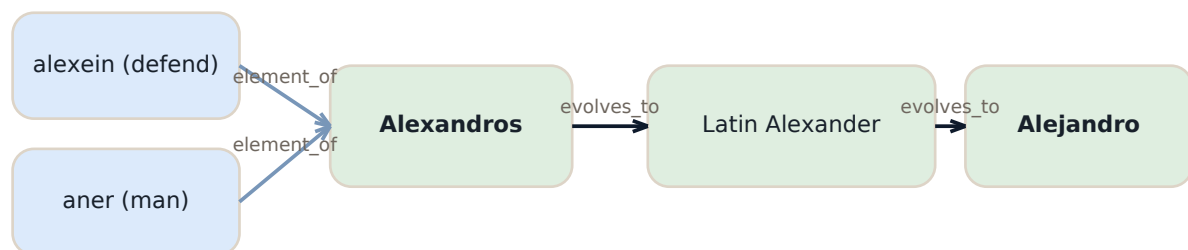
One-edge rule

There should be no more than one edge between the same two nodes, even if a pair could be described in more than one way. Choose the most useful relationship for the user and store nuance in the edge explanation or lineage summary.

Edge type	Direction	Use it for	Example
meaning_of	Concept -> Word	A concept carried by a word/root.	protection -> alexein
element_of	Word -> Name	A word/root contributes to a name.	alexein -> Alexandros
evolves_to	Name -> Name	Historical/transmission step.	Alexandros -> Latin Alexander
variant_of	Usually Name -> Name	Direct variant, short form, gendered form. Use sparingly.	Alexander -> Alex

Lineage paths

lineages.path_nodes uses nested arrays. Each inner array is one step group. Multiple nodes in the same group are parallel roots feeding into the next name. This is why alexein and aner sit in the first group.



path_nodes example: [[alexein, aner], [alexandros], [latin-alexander], [alejandro]]

```

-- Conceptual example, using readable labels here.
-- In the actual database, these are node UUID strings.
[
  ["node_alexein", "node_aner"],
  ["node_alexandros"],
  ["node_latin_alexander"],
  ["node_alejandro"]
]

```

Important: a lineage can include a sequence of nodes without creating shortcut family edges. For example, Iskandar can be shown as alexein + aner -> Alexandros -> Iskandar without also creating

Alexander -> Iskandar.

8. How the code queries the database

The current app uses a small wrapper called `db(table, params)`. It builds a Supabase REST URL, adds the `apikey` and `Authorization` headers, calls `fetch()`, and returns JSON rows. Each frontend query is just a table name plus PostgREST-style filters.

```
async function db(table, params = {}) {
  const response = await fetch(apiUrl(table, params), {
    headers: {
      "apikey": SUPABASE_KEY,
      "Authorization": `Bearer ${SUPABASE_KEY}`,
      "Content-Type": "application/json"
    }
  });

  if (!response.ok) {
    const text = await response.text();
    throw new Error(`Supabase error on ${table}: ${response.status} ${text}`);
  }

  return response.json();
}
```

Graph loading sequence

Step	Function	What happens
1	<code>loadGraphBySlug(slug)</code>	Finds the centre node in nodes by slug and optional node_type.
2	<code>loadGraphByNode(center)</code>	Fetches the primary lineage for the centre node.
3	<code>flattenPathNodes(path_nodes)</code>	Extracts all node IDs from the nested lineage array.
4	<code>db("edges")</code>	Fetches edges connected to those lineage/path node IDs.
5	<code>db("nodes")</code>	Fetches all nodes needed for the local graph.
6	<code>renderGraph()</code>	Converts rows into Cytoscape elements and applies styles.
7	<code>selectNode()</code>	Loads details from names, words, or concepts for the side panel.

Where to change behaviour

Desired change	Edit this function/area
Change graph layout positions	<code>makePositions()</code>
Change colours, line styles, node sizes	Cytoscape style array inside <code>renderGraph()</code>
Change what loads when searching	<code>runSearch()</code> and <code>loadGraphByNode()</code>
Add a side-panel field	<code>renderDetails()</code>
Add a new node type	<code>tableByType</code> in <code>loadDetailsForNode()</code> , styles, and database rules
Make clicking a node re-centre the graph	<code>cy.on("tap", "node", ...)</code> currently calls <code>selectNode()</code> ; change to <code>loadGraphByNode(node)</code> after fetching node row.

9. How to add a new name

Adding a new name should not require a frontend code change. Add the correct rows to the database and the website will pick them up through search and graph loading.

Data entry checklist

- Check whether the name, roots, concepts, and language already exist by slug.
- Add source rows first if they do not already exist.
- Add the node row for the new name.
- Add the names detail row linked by node_id.
- Add only the necessary edge(s). Do not create shortcut edges just because two names are related.
- Add a primary lineage with full path_nodes back to source roots where known.
- Test by searching for the new slug on the GitHub Pages site.
- If the graph looks too crowded, remove redundant edges before changing code.

SQL pattern

```
-- 1. Insert or reuse the node
INSERT INTO nodes (node_type, slug, display_label, canonical_key, description_short, created_at, updated_at)
VALUES ('name', 'new-name-slug', 'New Name', 'new name', 'Short description.', NOW(), NOW())
ON CONFLICT (node_type, slug) DO UPDATE
SET display_label = EXCLUDED.display_label,
    canonical_key = EXCLUDED.canonical_key,
    description_short = EXCLUDED.description_short,
    updated_at = NOW();

-- 2. Insert detail row linked to nodes.id
INSERT INTO names (node_id, display_name, primary_language_id, gender_usage,
    short_summary, long_summary, source_ids, created_at, updated_at)
VALUES (
    (SELECT id FROM nodes WHERE node_type = 'name' AND slug = 'new-name-slug' LIMIT 1),
    'New Name',
    (SELECT id FROM languages WHERE slug = 'language-slug' LIMIT 1),
    'masculine',
    'Short summary.',
    'Longer explanation.',
    (SELECT COALESCE(jsonb_agg(id::text), '[]'::jsonb)
     FROM sources WHERE title IN ('Source title here')),
    NOW(), NOW()
)
ON CONFLICT (node_id) DO UPDATE
SET short_summary = EXCLUDED.short_summary,
    long_summary = EXCLUDED.long_summary,
    source_ids = EXCLUDED.source_ids,
    updated_at = NOW();
```

Lineage SQL pattern

```
INSERT INTO lineages (node_id, lineage_type, title, summary, path_nodes,
    certainty, source_ids, created_at, updated_at)
VALUES (
    (SELECT id FROM nodes WHERE node_type = 'name' AND slug = 'new-name-slug' LIMIT 1),
    'primary',
    'Etymology of New Name',
    'Explain the full lineage in one sentence.',
    jsonb_build_array(
        jsonb_build_array(
            (SELECT id::text FROM nodes WHERE node_type = 'word' AND slug = 'root-1' LIMIT 1),
            (SELECT id::text FROM nodes WHERE node_type = 'word' AND slug = 'root-2' LIMIT 1)
        ),
        jsonb_build_array((SELECT id::text FROM nodes WHERE node_type = 'name' AND slug = 'older-form' LIMIT 1)),
        jsonb_build_array((SELECT id::text FROM nodes WHERE node_type = 'name' AND slug = 'new-name-slug' LIMIT 1))
    ),
    1,
    [],
    NOW(), NOW()
);
```

```

),
0.80,
(SELECT COALESCE(jsonb_agg(id::text), '[]'::jsonb)
 FROM sources WHERE title IN ('Source title here')),
NOW(), NOW()
)
ON CONFLICT (node_id, lineage_type, title) DO UPDATE
SET summary = EXCLUDED.summary,
    path_nodes = EXCLUDED.path_nodes,
    certainty = EXCLUDED.certainty,
    source_ids = EXCLUDED.source_ids,
    updated_at = NOW();

```

10. How to update the frontend code

The prototype is a single HTML file. That makes it easy to hand over, but the developer should avoid mixing unrelated changes. Update one section at a time and test after every change.

Code area	What it controls	Safe update guidance
CONFIG block	Supabase URL and publishable key.	Change only when moving to a different Supabase project. Never use service_role.
apiUrl(), db(), getSingle()	Low-level REST access.	Do not change unless Supabase API shape changes.
loadGraphBySlug()	Finds the centre node by slug.	Keep slug lookup stable. Add node_type only if needed.
loadGraphByNode()	Builds the local graph.	Change here if you want depth-2 expansion or RPC loading later.
makePositions()	Manual lineage-first layout.	This is the best place to improve visual layout.
renderGraph()	Cytoscape setup and style selectors.	Change node/edge visual design here.
selectNode()	Side-panel selection behaviour.	Currently selects node only; can be changed to re-centre graph.
loadDetailsForNode()	Chooses names/words/concepts detail table.	Update if new node types are added.
renderDetails()	What text appears in side panel.	Add new fields here.
runSearch()	Search dropdown.	Add ranking, aliases, or search_terms here later.

Recommended refactor later

Once the prototype is stable, split index.html into index.html, style.css, and app.js. After that, consider adding a Supabase RPC function such as get_local_graph(slug text) so the browser makes one clean request instead of several table requests.

11. Security and deployment notes

GitHub Pages is static hosting. Anything placed in index.html is public. The Supabase publishable key is acceptable in browser code only if RLS policies protect the tables properly. Do not expose secret/service-role keys in frontend code.

Topic	Current decision	Developer note
GitHub Pages	Deploy index.html from root or docs folder.	No server-side code or secret environment variables.
Supabase key	Use publishable key in browser.	RLS must be reviewed. Never use service_role.
Public read	Prototype tables are readable by anon role.	Only allow SELECT. Do not allow public insert/update/delete.
Write access	SQL Editor / Supabase dashboard only for now.	Later build an admin interface with authentication.
Data validation	Mostly manual SQL currently.	Later add check constraints, triggers, and admin forms.

Prototype public read policy pattern

```
-- Example read-only policy for one table.  
alter table nodes enable row level security;  
  
drop policy if exists "Public read nodes" on nodes;  
create policy "Public read nodes"  
on nodes for select  
to anon  
using (true);
```

12. Troubleshooting

Problem	Likely cause	Fix
Could not find node with slug	No row in nodes, slug typo, or RLS blocking read.	Run SELECT in SQL Editor; check public read policy; check slug lowercase.
No API key found in request	Testing REST URL directly without apikey header or URL param.	The app uses headers. For direct browser test, add &apikey;=... or use DevTools fetch.
Graph shows node but no details	Missing detail row in names/words/concepts.	Check SELECT * FROM names WHERE node_id = ...
Sources do not show	source_ids is empty or contains invalid IDs.	Store source_ids as JSONB array of source UUID strings.
Duplicate or messy lines	Too many shortcut edges or duplicate edge types.	Enforce one-edge rule and delete redundant family edges.
SQL says column notes does not exist	words table uses additional_notes, not notes.	Use additional_notes in words inserts.
SQL says primary_language does not exist	names table uses primary_language_id.	Use primary_language_id foreign key.
SQL says column now does not exist	Used NOW instead of NOW().	Use NOW() with brackets.
Search finds result but graph is sparse	The selected node has no primary lineage.	Add lineages row with path_nodes.
Graph layout looks odd	path_nodes order or grouping is wrong.	Put parallel roots in first inner array; put each stage in order.

13. Developer handover checklist

- Confirm the GitHub Pages site loads index.html and the Supabase publishable key is present.
- Confirm RLS allows SELECT for anon on nodes, edges, edge_types, names, words, concepts, lineages, languages, and sources.
- Run SELECT node_type, slug, display_label FROM nodes ORDER BY created_at DESC LIMIT 20 to confirm data exists.
- Search Alexander, Alejandro, Iskandar, Sikandar, Alex, Alexandra, Alessandro, Alasdair on the live site.
- Check that every important name has a detail row and a primary lineage.
- Review edges for redundancy: only one edge between two nodes and no family shortcut edges unless direct variant/short/gendered form.
- When adding a new name, add sources first, then nodes/detail, then essential edges, then lineage.
- Do not edit UUIDs manually unless recovering data. Use slug/title/code lookups in SQL.
- Keep broad story notes in names.long_summary/cultural_notes/literary_notes, not as extra graph nodes.
- Before changing visual code, test whether the issue is actually bad data.

14. References

Ref	Source	What it supports	URL
[1]	Supabase Docs - Data REST API	Supabase auto-generates a REST API from the database schema and supports browser access.	https://supabase.com/docs/guides/api

Ref	Source	What it supports	URL
[2]	Supabase Docs - API keys	Publishable keys can be used in frontend code, but access must be guarded by roles/RLS; secret keys must not be exposed.	https://supabase.com/docs/guides/getting-started/api-keys
[3]	Supabase Docs - Row Level Security	RLS policies provide protection for data accessed from browser-facing clients.	https://supabase.com/docs/guides/database/postgres/row-level-security
[4]	GitHub Docs - GitHub Pages publishing source	GitHub Pages can publish static files from a branch and root/docs folder.	https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site
[5]	GitHub Docs - Creating a GitHub Pages site	GitHub Pages publishes static files pushed to a repository.	https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site
[6]	Cytoscape.js documentation	Cytoscape.js is an open-source JavaScript graph theory/network visualisation library.	https://js.cytoscape.org/

End of guide.